

No. Numpy

[What is Numpy?](#)

[Installation of NumPy?](#)

[Checking NumPy Version?](#)

[How do I access or Slicing elements in a NumPy array?](#)

[numpy.insert](#)

Ques. What is Numpy?

- NumPy is short for "**Numerical Python**".
- NumPy is a Python library. it is used for working with arrays.
- NumPy was created in **2005** by **Travis Oliphant**.

Installation of NumPy?

- If you have Python and PIP already installed on a system, then installation of NumPy is very easy.

```
# Open cmd
C:\Users\Your Name> pip install numpy
```

Ques. Checking NumPy Version?

```
import numpy as np
print(np.__version__)
```

Output:- 1.16.3

Example:-

- Once NumPy is installed, import it in your applications by adding the import keyword:

```
import numpy
arr = numpy.array([1, 2, 3, 4, 5])
print(arr) # Output:- [1 2 3 4 5]

# Create an **alias** with the as keyword while importing:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr) # Output:- [1 2 3 4 5]
```

Access Array/Array Slicing

How do I access or Slicing elements in a NumPy array?

- Slicing in python means taking elements from one given index to another given index.
- We pass slice instead of index like this: **[start:end]**.
- We can also define the step, like this: **[start : end : step]**.
- If we don't pass start its considered 0
- If we don't pass end its considered length of array in that dimension
- If we don't pass step its considered 1

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[0])          # output:- 1
print(arr[1])          # output:- 2
print(arr[2] + arr[3])  # Output:- 4+3=7

print(arr[1:5])         # Output:- [2 3 4 5]
print(arr[4:])          # Output:- [5 6 7]
print(arr[:4])          # Output:- [1 2 3 4]
print(arr[-3:-1])       # Output:- [5 6]
print(arr[1:5:2])       # Output:- [2 4]
print(arr[:,2])         # Output:- [1 3 5 7]

# -----Slicing 2-D Arrays-----
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0, 1])        # Output:- 2
print(arr[1, 4])        # Output:- 10
print(arr[1, -1])       # Output:- 10    Negative index

print(arr[1, 1:4])       # Output:- [7 8 9]
print(arr[0:2, 2])       # Output:- [3 8]
print(arr[0:2, 1:4])     # Output:- [[2 3 4][7 8 9]]

# -----Access 3-D Arrays-----
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])      # Output:- 6
```

Ques. Why is NumPy Faster Than Lists?

Creating Arrays

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)          # Output:- [1 2 3 4 5]
print(type(arr))    # Output:- <class 'numpy.ndarray'>

# 0-D Arrays
import numpy as np
arr = np.array(42)
print(arr)  # Output:- 42

# 1-D Arrays
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)  # Output:- [1 2 3 4 5]

# 2-D Arrays
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)  # Output:- [[1 2 3][4 5 6]]

# 3-D arrays
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)    # Output:- [[[1 2 3][4 5 6]] [[1 2 3] [4 5 6]]]
```

How to Check Number of Dimensions?

- NumPy Arrays provides the **ndim attribute** that returns an integer that tells us how many dimensions the array have.

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)  # Output:- 0
print(b.ndim)  # Output:- 1
print(c.ndim)  # Output:- 2
print(d.ndim)  # Output:- 3
```

Higher Dimensional Arrays

- When the array is created, we can define the number of dimensions by using the **ndmin** argument.

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)  # Output:- [[[[[1 2 3 4]]]]]
print('number of dimensions :', arr.ndim)  # Output:- number of dimensions : 5
```

NumPy Array Copy vs View

- The **main difference** between a **copy** and a view of an array is that the copy is a **new array**, and the **view** is just a view of the **original array**.

Copy

- The **copy** owns the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)  # Output:- [42  2  3  4  5]
print(x)    # Output:- [1 2 3 4 5]
```

View

- The **view** does not own the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42

print(arr)  # Output:- [42  2  3  4  5]
print(x)    # Output:- [42  2  3  4  5]
```

How to Check if Array Owns its Data

- Every NumPy array has the attribute `base` that returns `None` if the array owns the data.
- Otherwise, the `base` attribute refers to the original object.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()
y = arr.view()

print(x.base)    # Output:- None
print(y.base)    # Output:- [1 2 3 4 5]
```

numpy.insert

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Insert 25 at index 2
new_arr = np.insert(arr, 2, 25)
print("After inserting 25 at index 2:", new_arr)    # Output:- [10 20 25 30 40 50]

# # Insert [9, 10] at index 3
new_arr = np.insert(arr, 3, [9, 10])
print("After inserting [9,10] at index 3:", new_arr) # Output:- [10 20 30  9 10 40
50]
```

Explanation:

- The second argument (2) is the index where you want to insert the value.
- The third argument (25) is the value you want to insert.
- The original array remains unchanged; np.insert() returns a new array.
- **Inserting multiple values at different positions:**

```
arr = np.array([10, 20, 30, 40, 50])
# Insert values 15 and 35 at indices 1 and 3
new_arr = np.insert(arr, [1, 3], [15, 35])
print("After inserting [15, 35] at indices [1, 3]:", new_arr)    # Output:- [10 15
20 30 35 40 50]
```

Insert in a 2D array along a specific axis

```
import numpy as np

arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])

# Insert a new row [9, 9, 9] at index 1 (between rows 0 and 1)
new_arr2d = np.insert(arr2d, 1, [9, 9, 9], axis=0)    # axis 0 means row
print("After inserting new row at index 1:\n", new_arr2d)

# Output:-
After inserting new row at index 1:
[[1 2 3]
 [9 9 9]
 [4 5 6]]
```

```
import numpy as np

arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])

# Insert a new column [7, 8] at index 2 (between columns 1 and 2)
new_arr2d_col = np.insert(arr2d, 2, [7, 8], axis=1) # axis 1 means column
print("After inserting new column at index 2:\n", new_arr2d_col)

# Output:-
After inserting new column at index 2:
[[1 2 7 3]
 [4 5 8 6]]
```

- insert element end of the list

```
import numpy as np

arr = np.array([5, 6, 7])

# Insert at the end (index = len(arr))
arr_end = np.insert(arr, len(arr), 10)
print("Insert 10 at end:", arr_end) # Output:- [ 5  6  7 10]
```

import time

```
start = time.time() for i in range(1000000): # Reduced the range for demonstration mul = i ** 2
print("Execution time:", time.time() - start)
```

import time

```
start = time.time() squares = [i ** 2 for i in range(1000000)] # Reduced the range for demonstration
print("Execution time:", time.time() - start)
```

```
import time import numpy as np start = time.time()
```

```
squares = np.arange(1000000) ** 2 execution_time = time.time() - start print(execution_time)
```

Creating array from scratch

Creating Arrays with Zeros :- creates an array filled with zeros. 1-D Array of Zeros: import numpy as np

```
zeros_1d = np.zeros(5) # Creates a 1-D array with 5 zeros print("1-D Array of Zeros:", zeros_1d)
```


2-D Array of Zeros: `zeros_2d = np.zeros((3, 4))` # Creates a 2-D array with shape (3, 4) filled with zeros
`print("2-D Array of Zeros:\n", zeros_2d)`

3-D Array of Zeros: `zeros_3d = np.zeros((2, 3, 4))` # Creates a 3-D array with shape (2, 3, 4) filled with zeros
`print("3-D Array of Zeros:\n", zeros_3d)`

Creating Arrays with Ones

- creates an array filled with ones. 1-D Array of Ones: `ones_1d = np.ones(5)` # Creates a 1-D array with 5 ones
`print("1-D Array of Ones:", ones_1d)`

2-D Array of Ones: `ones_2d = np.ones((3, 4))` # Creates a 2-D array with shape (3, 4) filled with ones
`print("2-D Array of Ones:\n", ones_2d)`

3-D Array of Ones: `ones_3d = np.ones((2, 3, 4))` # Creates a 3-D array with shape (2, 3, 4) filled with ones
`print("3-D Array of Ones:\n", ones_3d)`

`np.full()` Creates an array filled with a specified value.

`full_array = np.full((2, 3), 7)` # Creates a 2x3 array filled with the value 7
`print("Full Array:\n", full_array)`

[||How do you calculate the dot product of two NumPy arrays?|](#)

How do you calculate the dot product of two NumPy arrays?

1. Using np.dot()

- The np.dot() function computes the dot product of two arrays. For 1-D arrays, it is the inner product of the vectors. For 2-D arrays, it performs matrix multiplication.

```
import numpy as np

# Define two 1-D arrays (vectors)
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate the dot product
dot_product = np.dot(a, b)
print("Dot Product using np.dot():", dot_product) # Output:- Dot Product using
np.dot(): 32
```

2. Using the @ Operator

- In Python 3.5 and later, you can use the @ operator to perform matrix multiplication, which also calculates the dot product for vectors.

```
import numpy as np

# Define two 1-D arrays (vectors)
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate the dot product
dot_product = a @ b
print("Dot Product using @:", dot_product) # Output:- Dot Product using @: 32
```

3. Using np.matmul()

```
import numpy as np

# Define two 1-D arrays (vectors)
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate the dot product
dot_product_matmul = np.matmul(a, b)
print("Dot Product using np.matmul():", dot_product_matmul) # Output:- Dot Product
using np.matmul(): 32
```

Complete Example

```
import numpy as np

# Define two 1-D arrays (vectors)
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Method 1: Using np.dot()
dot_product1 = np.dot(a, b)
print("Dot Product using np.dot():", dot_product1) # Output:- Dot Product using
np.dot(): 32

# Method 2: Using @ operator
dot_product2 = a @ b
print("Dot Product using @ operator:", dot_product2) # Output:- Dot Product using
@ operator: 32

# Method 3: Using np.matmul()
dot_product3 = np.matmul(a, b)
print("Dot Product using np.matmul():", dot_product3) # Output:- Dot Product using
np.matmul(): 32
```